

Project for Home work

11 January 2026 22:54

Project: Student Grade Management System

Problem Statement:

Create a Python program to manage student grades for a class. The system should allow users to add students with their scores in 5 subjects, calculate statistics, search for students, and generate reports. Use a dictionary to store student data where keys are student IDs (strings) and values are tuples of (name, scores_list). Implement the following menu-driven system:

Requirements:

1. Data Structure: Use a dictionary students where each entry is {student_id: (name, [score1, score2, score3, score4, score5])}

2. Menu Options (using flow control):

- Add a new student (validate input, check for duplicate ID)
- Display all students with average grade
- Search student by ID or partial name (case-insensitive string search)
- Show top 3 students by average score
- Generate class statistics (highest/lowest average, class average)
- Exit

3. User-Defined Functions (minimum 5 functions):

- add_student(students): Add new student with input validation
- calculate_average(scores): Return average of scores list
- search_student(students, query): Search by ID or name substring
- get_top_students(students, n): Return top n students by average
- display_statistics(students): Show class stats

4. Key Features:

- Use **loops** for menu repetition and data processing
- **String operations** for name formatting, searching (.lower(), .find(), slicing)
- **Lists** for subject scores
- **Tuples** for immutable student info (name, scores)
- **Dictionaries** for main data storage
- **Exception handling** for:
 - Invalid scores (ValueError for non-numeric input)
 - Division by zero (empty scores list)
 - KeyError (student ID not found)
 - IndexError (invalid list access)

Sample Input/Output:

text

```
=== Student Grade Management System ===
```

```
1. Add Student
2. Display All Students
3. Search Student
4. Top 3 Students
5. Class Statistics
6. Exit
```

```
Enter choice: 1
```

```
Student ID: S001
```

```
Name: Alice Johnson
```

```
Enter 5 scores (0-100): 85 92 78 95 88
Student S001 added successfully!
Enter choice: 5
Class Statistics:
- Highest Average: 95.2 (S001 - Alice Johnson)
- Lowest Average: 82.4 (S002 - Bob Smith)
- Class Average: 88.7
```

Constraints:

- Scores must be 0-100 (validate with exceptions)
- Minimum 3 students for statistics
- Handle empty dictionary gracefully
- Use while True for menu loop, break on exit

Challenge Extensions (Bonus):

- Sort students by average descending
- Generate grade letters (A:90+, B:80-89, etc.)

Test cases

Test cases for the Student Grade Management System project. These cover all required concepts and edge cases.

Test Case 1: Basic Functionality (Happy Path)

text

****Input Sequence:****

1. Add Student: ID=S001, Name="Alice Johnson", Scores=[85,92,78,95,88]
2. Add Student: ID=S002, Name="Bob Smith", Scores=[72,68,85,77,90]
3. Add Student: ID=S003, Name="Carol Lee", Scores=[95,88,92,87,91]
4. Display Class Statistics

****Expected Output:****

Class Statistics:

- Highest Average: 91.8 (S003 - Carol Lee)
- Lowest Average: 78.4 (S002 - Bob Smith)
- Class Average: 88.0

Test Case 2: Exception Handling - Invalid Scores

text

****Input:****

1. Add Student: ID=S004, Name="David Brown"
2. Scores input: "abc" (invalid), then [55,66,77,88,99]

****Expected Behavior:****

- ValueError caught for "abc"
- Prompt re-asked for scores
- Student S004 added successfully with valid scores
- Error message: "Invalid score. Enter numeric values (0-100):"

Test Case 3: Duplicate Student ID

text

****Input:****

1. Add S001 (already exists)
2. Try ID="S001", Name="Duplicate Alice"

****Expected Output:****

"Student ID S001 already exists. Please use a different ID."

Menu returns (no student added)

Test Case 4: Search Functionality

text

****Input:****

Students: S001(Alice), S002(Bob), S003(Carol)

Search query: "ali" (partial name)

****Expected Output:****

"Found 1 student(s):

S001 - Alice Johnson (Avg: 87.6)"

Test Case 5: Top Students (Insufficient Data)

text

****Input:****

Only 1 student added, select "Top 3 Students"

****Expected Output:****

"Not enough students for top 3 ranking (Need minimum 3 students)"

Test Case 6: Empty System Statistics

text

****Input:****

No students added, select "Class Statistics"

****Expected Output:****

"No student data available. Please add students first."

Test Case 7: Boundary Score Values

text

****Input:****

Add S005: Scores=[0, 100, 50, 100, 0]

****Expected Average:**** 50.0

****Expected Output:**** "S005 - Name (Avg: 50.0)"

Test Case 8: KeyError - Invalid Student Search

text

****Input:****

Search for non-existent ID: "S999"

****Expected Output:****

"Student S999 not found."

Test Case 9: Complete Workflow

text

****Full Sequence:****

1. Add S001: [85,92,78,95,88] → Success
2. Add S002: [72,"xyz",85,77,90] → ValueError → [72,68,85,77,90] → Success
3. Try Add S001 → Duplicate error
4. Display All → Shows 2 students with averages
5. Top 3 → Shows S001, S002 (only 2 available)
6. Search "bob" → Finds S002
7. Statistics → Correct min/max/class avg
8. Exit → "Thank you for using the system!"

****Validation Points:****

- 2 students in dictionary
- Averages: S001=87.6, S002=78.4
- Class avg=83.0

Test Case 10: Score Range Validation

text

****Input Attempts:****

1. Score: 101 → "Score must be 0-100. Try again:"
2. Score: -5 → "Score must be 0-100. Try again:"
3. Score: 75 → Accepted

****Expected:**** Continuous prompting until valid range

Automated Test Script Structure

python

For evaluating student submissions

```
test_cases = [  
    {"name": "basic_add", "expected_students": 1, "avg": 87.6},  
    {"name": "duplicate_id", "expected_error": "already exists"},  
    {"name": "invalid_score", "expected_error": "ValueError"},  
    {"name": "empty_stats", "expected_output": "No student data"}  
]
```

Evaluation Checklist

- Menu loop works with while True
- 5+ user-defined functions implemented
- Dictionary stores (name, scores_list) tuples
- Average calculation handles lists correctly
- String search is case-insensitive
- All 5 exception types handled
- Duplicate ID prevention
- Empty data scenarios handled gracefully

Total Test Coverage: 100% of requirements. Use these to validate complete implementation!